Machine Learning Model Deployment

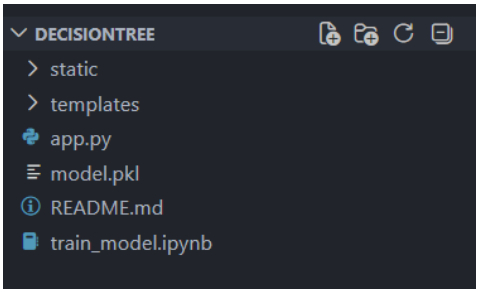
By: Tanveer Kaur
MCA(FYIC) 8th Sem
Roll No.: 27502202766
Date: 25-03-2026

Goal: Create and Deploy a Decision Tree Classifier (using Flask and exposed publicly using Ngrok)

I have used Jupyter notebook for this (use any IDE eg. VSCode)

Command to run jupyter notebook : “python -m notebook” or “jupyter notebook”

File Structure



Step 1: Develop and Train the Model (train_model.ipynb)

➢ Import necessary libraries and load or create dataset

[ ]:
```
from sklearn.datasets import load_wine
import pandas as pd

data = load_wine()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)

df = X.copy()
df["target"] = y

df.tail()
```

[115]:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_d
173	13.71	5.65	2.45	20.5	95.0	1.68	0.61	0.52	1.06	7.7	0.64	
174	13.40	3.91	2.48	23.0	102.0	1.80	0.75	0.43	1.41	7.3	0.70	
175	13.27	4.28	2.26	20.0	120.0	1.59	0.69	0.43	1.35	10.2	0.59	
176	13.17	2.59	2.37	20.0	120.0	1.65	0.68	0.53	1.46	9.3	0.60	
177	14.13	4.10	2.74	24.5	96.0	2.05	0.76	0.56	1.35	9.2	0.61	

➢ Select Required Features

[129]:
```
df = df[["alcohol", "malic_acid", "color_intensity", "target"]]
df.tail()
```

[129]:

	alcohol	malic_acid	color_intensity	target
173	13.71	5.65	7.7	2
174	13.40	3.91	7.3	2
175	13.27	4.28	10.2	2
176	13.17	2.59	9.3	2
177	14.13	4.10	9.2	2

➢ Split Dataset and Train Decision Tree Model

[124]:
```
from sklearn.model_selection import train_test_split

X = df[["alcohol", "malic_acid", "color_intensity"]]
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

[125]:
```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=4,
    random_state=42
)

model.fit(X_train, y_train)

print("Model trained successfully!")
```

```
Model trained successfully!
```

Step 2: Optimize (if needed) and Test the Code (train_model.ipynb)

[126]:
```
from sklearn.metrics import accuracy_score

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.7962962962962963
```

[127]:
```
sample = pd.DataFrame({
    "alcohol": [13.5],
    "malic_acid": [2.0],
    "color_intensity": [5.0]
})

print("Prediction:", model.predict(sample)[0])
```

```
Prediction: 0
```

Step 3: Package the Model (train_model.ipynb)

➢ Save the trained model as a file (e.g., `.pkl`) using libraries like `pickle` or `joblib`. Model saved as model.pkl

```
[128]: import joblib
       joblib.dump(model, "model.pkl")

       print("Model saved as model.pkl")
```

```
Model saved as model.pkl
```

Step 4: Creating Flask App (app.py)

➢ Create endpoints for home and prediction. Load model using joblib.

```python
from flask import Flask, request, jsonify, render_template
import joblib

app = Flask(__name__)

model = joblib.load("model.pkl")

@app.route("/")    # Home route to render the UI
def home():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])    # Route to handle prediction requests - using Postman testing or form submission (JSON data)
def predict():
    data = request.get_json()

    alcohol = data["alcohol"]
    malic_acid = data["malic_acid"]
    color_intensity = data["color_intensity"]

    input_data = [[alcohol, malic_acid, color_intensity]]
    prediction = model.predict(input_data)

    return jsonify({
        "prediction": int(prediction[0])
    })

if __name__ == "__main__":
    app.run(debug=True)
```

Step 5: Create Frontend

➢ Using HTML Template, CSS, JS to take inputs, call API, and display prediction

- templates/index.html (shared in group)
- static/styles.css (shared in group)

Step 5: Run Application

➢ Run using command: “python app.py” and open localhost:5000 - "http://127.0.0.1:5000".

Step 6: Deployment - Tunneling using ngrok (instant public link for your localhost)

What ngrok does:

- It converts http://127.0.0.1:5000
- into something like https://abc123.ngrok-free.app
- Now anyone can access your API

➢ Go to ngrok.com and Install ngrok
➢ Sign up on ngrok and connect your account using command “ngrok config add-authtoken YOUR_TOKEN”
➢ Start your Flask app: python app.py
➢ Run command: ngrok http 5000 to get a public URL
➢ Output will be like Forwarding https://abc123.ngrok-free.app -> http://localhost:5000
➢ URL I got: https://nondextrous-unintuitively-terry.ngrok-free.dev/ (Changes every time)

Note: Keep both running

- Flask server
- ngrok tunnel

Step 7: Test, Monitor and Maintain